

MODULE-2

Chapter-1: The Relational Data Model and Relational Database Constraints

Reference: Fundamentals of Database Systems by Elmasri and Navathe

CONTENTS

- **Relational Model Concepts**
- **Relational Model Constraints**
- **Relational Database Schemas**

RELATIONAL MODEL CONCEPTS

- A Relation is a mathematical concept based on the ideas of sets.
- The model was first proposed by Dr. E.F. Codd of IBM Research in 1970 in the following paper:

"A Relational Model for Large Shared Data Banks,"

Communications of the ACM, June 1970

- The above paper caused a major revolution in the field of database management and earned Dr. Codd the coveted ACM Turing Award.

RELATIONAL MODEL CONCEPTS

Informal Definitions:

- A **relation** looks like a **table** of values.
- A relation typically contains a **set of rows**.
- The data elements in each **row** represent certain facts that correspond to a real-world **entity** or **relationship**.
- Each **column** has a column header that gives an indication of the meaning of the data items in that column.
- **Key of a Relation:** Each row has a value of a data item (or set of items) that uniquely identifies that row in the table, called the key.

*E.g.: In the **STUDENT** table, **USN** is the key*

RELATIONAL MODEL CONCEPTS

Formal Definitions- Schema of a Relation:

- Description of the Relation.
- Denoted as $R(a_1, a_2, \dots, a_n)$, where **R** is the name of the relation and **a1, a2, ..., an** are the attributes of the relation.
- Eg: CUSTOMER (Cust-id, Cust-name, Address, Phone#)
- Each attribute has a domain or a set of valid values.

RELATIONAL MODEL CONCEPTS

Formal Definitions- Tuple

- The rows of the relation are called as **tuples**. A relation is a set of such tuples.
- **Tuple** is an ordered set of values (enclosed in angled brackets ‘< ... >’)
- Each value is derived from an appropriate *domain*.
- A row in the **CUSTOMER** relation is a 4-tuple and would consist of four values, for eg.: <632895, "John Smith", "101 Main St. Atlanta, GA 30332", "(404) 894-2000">

This is called a 4-tuple as it has 4 values in the CUSTOMER relation.

RELATIONAL MODEL CONCEPTS

Formal Definitions- Domain

- A **domain** has a logical definition.

Eg.: **Employee_DoB**

- A domain also has a data-type or a format defined for it.

Eg.: **Employee_DoB** is of **Date** data type. *Dates have various formats such as year, month, date, formatted as **yyyy-mm-dd**, or as **dd-mm-yyyy** etc. Therefore, **Employee_DoB Date** is the format to define it.*

- The attribute name designates the role played by a domain in a relation.
- It is used to interpret the meaning of the data elements corresponding to that attribute.

RELATIONAL MODEL CONCEPTS

Formal Definitions- States

- The **relation state** is a subset of the Cartesian product of the domains of its attributes. i.e., **relation state**: $r(R) \subset \text{dom}(A1) \times \text{dom}(A2)$.
- Each domain contains the set of all possible values the attribute can take.
- Eg.: *Cust-name varchar(25)*

Here, the attribute **Cust-name** is of data type **string** and defined over the domain of character strings of maximum **length 25**.

- The role these strings play in the **CUSTOMER** relation is that of the name of a customer.

EXAMPLE OF A RELATION, ITS ATTRIBUTES AND TUPLES

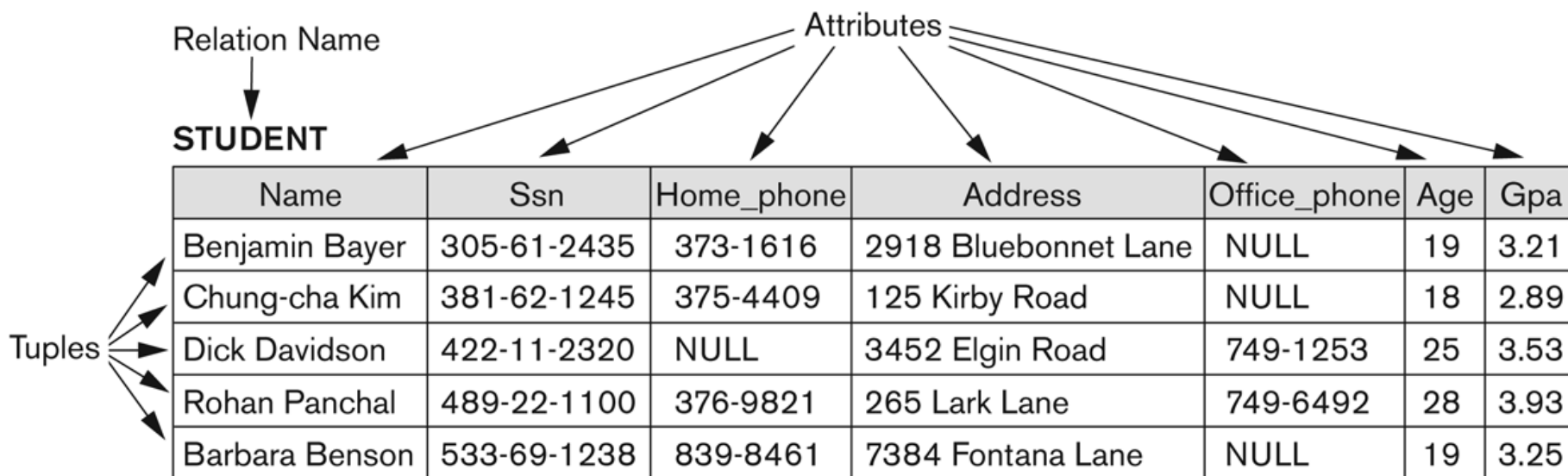


Figure 5.1

The attributes and tuples of a relation STUDENT.

DEFINTION SUMMARY

<u>Informal Terms</u>	<u>Formal Terms</u>
Table	Relation
Column Header	Attribute
All possible Column Values	Domain
Row	Tuple
Table Definition	Schema of a Relation
Populated Table	State of the Relation

CHARACTERISTICS OF RELATIONS

- **Data Representation:** Data is organized in tables (relations), with rows (tuples) representing records and columns (attributes) representing data fields.
- **Atomic Values:** Each attribute in a table contains atomic values, meaning no multi-valued or nested data is allowed in a single cell.
- **Unique Keys:** Every table has a primary key to uniquely identify each record, ensuring no duplicate rows.
- **Attribute Domain:** Each attribute has a defined domain, specifying the valid data types and constraints for the values it can hold.

CHARACTERISTICS OF RELATIONS

- **Data Independence:** The model ensures logical and physical data independence, allowing changes in the database schema without affecting the application layer.
- **Relational Operations:** Supports operations like selection, projection, join, union and intersection, enabling powerful data retrieval manipulation.
- **Data Consistency:** Ensures data consistency through constraints, reducing redundancy and anomalies.
- **Set-Based Representation:** Tables in the relational model are treated as sets and operations follow mathematical set theory principles.

RELATIONAL MODEL NOTATION

- Relation schema R of degree n is denoted by $R(A_1, A_2, \dots, A_n)$.
- Uppercase letters Q, R, S denote relation names.
- Lowercase letters q, r, s denote relation states.
- Letters t, u, v denote tuples.
- In general, the name of a relation schema such as **STUDENT** also indicates the current set of tuples in that relation.

RELATIONAL MODEL NOTATION

- An attribute A can be qualified with the relation name R to which it belongs by using the dot notation $R.A$.

Eg.: STUDENT.Name or STUDENT.Age.

- An n -tuple t in a relation $r(R)$ is represented as $t = \langle v_1, v_2, \dots, v_n \rangle$ where v_i is the value corresponding to the attribute A_i . The value v_i for attribute A_i in tuple t can be accessed using $t[A_i]$ or $t.A_i$.

RELATIONAL INTEGRITY CONSTRAINTS

- Constraints are **conditions** that must hold on **all** valid relation states.
- There are three *main types* of constraints in the relational model:

***Key** constraints*

***Entity integrity** constraints*

***Referential integrity** constraints*

- Another implicit constraint is the **domain** constraint

*Every value in a tuple must be from the domain of its attribute (or it could be **null**, if allowed for that attribute).*

TYPES OF KEYS

- **Candidate Key:** A table can have multiple candidate keys which looks to be unique out of which one can be selected as a primary key.

Eg.: CAR(Engine_Chassis_no, Registration_no, model, color)

Here Engine_Chassis_no and Registration_no are considered as candidate keys because both attributes looks logically unique.

- **Primary Key:** it is one of the candidate key currently considered as a key to identify each record uniquely. Primary key attributes are underlined. It can not have NULL values.

In the above eg., either Engine_Chassis_no or Registration_no can be considered as a primary key.

TYPES OF KEY CONSTRIANTS

- **Alternate Key:** it is a candidate key which is currently not selected as a primary key.

In the above eg., if **Engine_Chassis_no** is selected as a primary key, then **Registration_no** can be considered as an alternate keyf **Registration_no** is selected as a primary key, then **Engine_Chassis_no** can be considered as an alternate key.

- **Unique Key:** it is similar to primary key but with one variation that, it can contain NULL values in the column.

TYPES OF KEY CONSTRIANTS

- **Super Key:** Set of attribute/columns that uniquely identifies each row in the table upon which all other attributes of the table are functionally dependent.
- Primary key and candidate keys are called as minimal super keys/ subset of super key.

Eg.: **EMPLOYEE(Eid, Ename, Gender, Addr, Contact)**

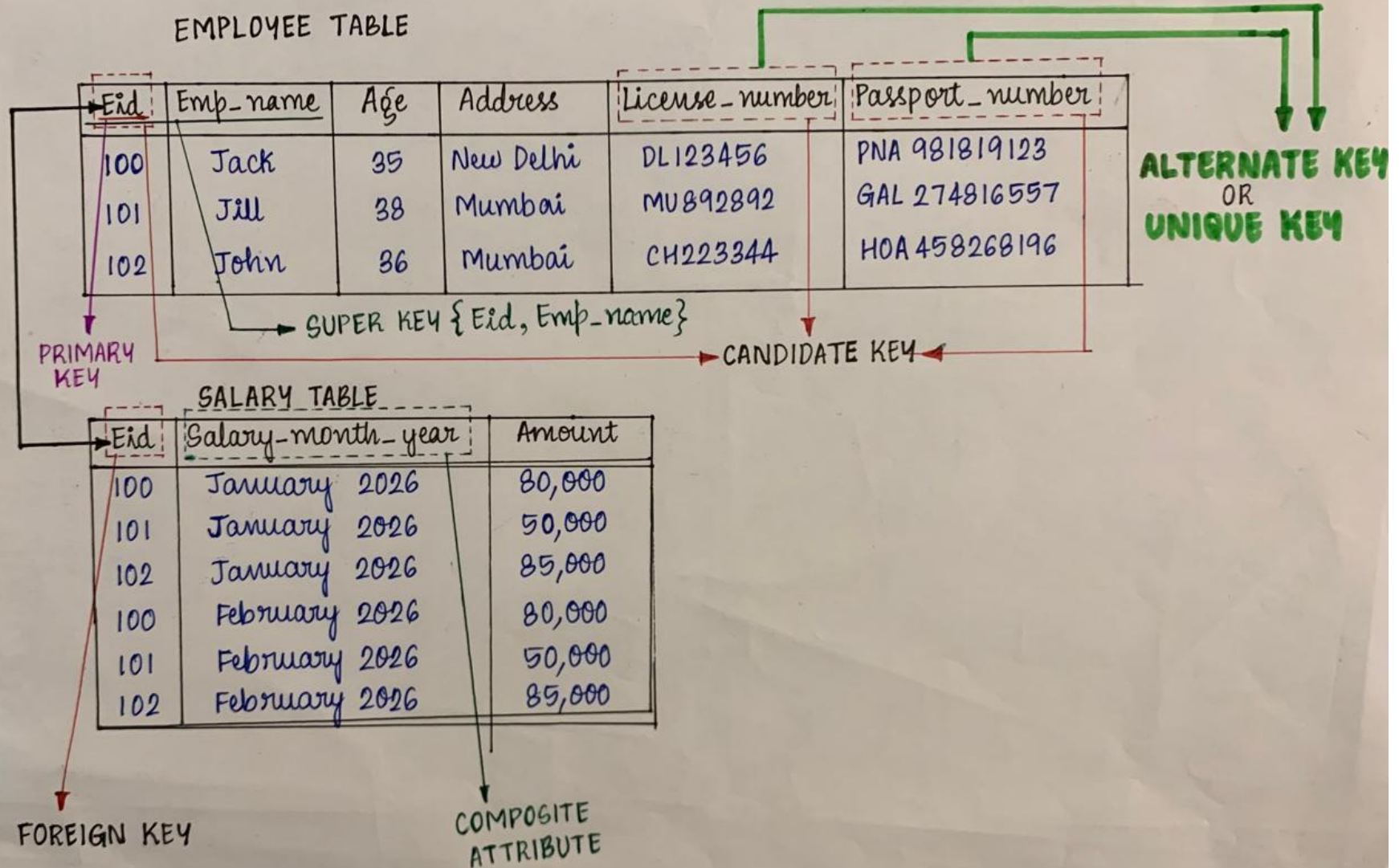
Here if **Eid** is a primary key, then any set of attributes combined with

Eid becomes super key like **{Eid, Ename}, {Eid, Ename, Addr}**

TYPES OF KEY CONSTRIANTS

- **Foreign Key:** A primary key of one table used as a reference in another table, that occurrence is called as foreign key.
- It can have repeated values of primary key values.
- **Composite Key:** it is a compound key- group of two or more attributes that identifies each row of the table uniquely.
- **Partial Key:** the key attribute of a weak entity (any attribute appealing to be unique) can be considered as a partial key

EXAMPLE OF TYPES OF KEYS



RELATIONAL DATABASE SCHEMA

- Relational database schema is a set of relation schemas that belong to the same database.
- It is denoted as, $S = \{R_1, R_2, \dots, R_n\}$ where *S is the name of the whole database schema and $R_1, R_2, \dots, R_n$ are the names of the individual relation schemas within the database S.*
- Eg.: Consider a COMPANY DATABASE with the following Schema.

EMPLOYEE(Fname, Minit, Lname, SSN, Bdate, Address, Sex, Salary, Super_ssn, Dno)

DEPARTMENT(Dname, Dnumber, Mgr_ssn, Mgr_start_date)

DEPT_LOCATIONS(Dnumber, Dlocation)

PROJECT(Pname, Pnumber, Plocation, Dnum)

WORKS_ON(Essn, Pno, Hours)

DEPENDENT(Essn, Dependent_name, Sex, Bdate, Relationship)

Here, $S = \{\text{EMPLOYEE, DEPARTMENT, DEPT_LOCATIONS, PROJECT, WORKS_ON, DEPENDENT}\}$, S is COMPANY DATABASE having 6 relation schemas.

SCHEMA DIAGRAM

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------

Figure 5.5
Schema diagram for
the COMPANY
relational database
schema.

INTEGRITY CONSTRAINTS

- Integrity constraints are the set of rules used to ensure that the data in a database is accurate, consistent and reliable.
- They act like a set of guidelines that ensure all the information stored in the database follows specific standards.
- These rules helps in maintaining the quality of data by ensuring that the processes like adding, updating or deleting information do not harm the integrity of the database.

Types of Integrity Constraints

- Domain Constraints
- Entity Integrity Constraints
- Referential integrity constraints
- Key Constraints

TYPES OF CONSTRAINTS

Domain Constraints

- Domain constraints are a type of integrity constraint that ensure the values stored in a column (or attribute) of a database are valid and within a specific range or domain.
- In simple terms, they define what type of data is allowed in a column and restrict invalid data entry.
- The data type of domain include string, char, time, integer, date, currency etc.

TYPES OF CONSTRAINTS

Types of Domain Constraints:

- **NOT NULL Constraint:** Ensures no records can have NULL value.
- **CHECK Constraint:** This Constraint checks for any specified condition over any attribute.

Why Domain Constraints are Important ?

They prevent invalid or inconsistent data from entering the database.

They ensure the database is reliable and follows predefined business rules.

They make the database easier to manage and maintain by reducing errors.

Example of Domain Constraint

Student_Id	Name	Semester	Age
21CSE100	Aniket Kumar	6th	20
21CSE101	Shashwat Dubey	7th	21
21CSE102	Manvendra Sharma	8th	22
21CSE103	Ashmit Dubey	5th	20

1.Student_Id: Must be unique and follow a specific format like 21CSE####. No duplicates or invalid formats allowed.

2.Name: Accepts only valid text (no numbers) and cannot be left empty (**NOT NULL constraint**).

3.Semester: Allows specific values like 5th, 6th, etc., and ensures valid input (e.g., no 10th if not permitted).

4.Age: Must be an integer within a reasonable range (e.g., 18-30) and cannot contain invalid data like negative numbers or text.

TYPES OF CONSTRAINTS

Entity Integrity Constraints

Entity integrity constraints state that primary key can never contain null value because primary key is used to determine individual rows in a relation uniquely, if primary key contains null value, then we cannot identify those rows. A table can contain null value in it except primary key field.

Key Features of Entity Integrity Constraints:

- **Uniqueness:** The primary key value must be unique for each row in the table. No duplicate entries are allowed in the primary key column.
- **NOT NULL:** The primary key column cannot contain NULL values, as every row must have a valid identifier.
- **Essential for Table Design:** Ensures that every record in the table can be uniquely identified, preventing ambiguity.

Example of Entity Integrity Constraint

Student_id	Name	Semester	Age
21CSE101	Ramesh	5th	20
21CSE102	Kamlesh	5th	21
21CSE103	Aakash	5th	22
NULL	Mukesh	5th	20

Here, the last entry of Student_id (primary key) is not allowed because it violates the rule of not containing NULL value.

Example of Entity Integrity Constraint

Student_id	Name	Semester	Age
21CSE1001	Sonali Rao	5th	20
21CSE1012	Anjali Gupta	5th	21
21CSE1023	Aastha Singh	5th	22
21CSE1034	Ayushi Singh	NULL	20

Here, let the not-null constraint be specified on the *Semester* attribute in the relation/table given above, then the data entry of 4th tuple will violate this integrity constraint, because the *Semester* attribute in this tuple contains null value. To make this database instance a legal instance, its entry must not be allowed by database management system.

TYPES OF CONSTRAINTS

Key Constraints

- Key constraints ensure that certain columns or combinations of columns in a table uniquely identify each row.
- These rules are essential for maintaining data integrity and preventing duplicate or ambiguous records.

Why Key Constraints Are Important ?

Prevent Duplicates: Ensure unique identification of rows.

Maintain Relationships: Enable proper linking between tables (via foreign keys).

Enforce Data Integrity: Prevent invalid or inconsistent data.

Example of Key Constraint

Student_id	Name	Semester	Age
21CSE101	Ramesh	5th	20
21CSE102	Kamlesh	5th	21
21CSE103	Aakash	5th	22
21CSE102	Mukesh	5th	20

All the entries in the table are valid because all rows are unique. Here *Student_id* is considered as the key attribute which takes only unique values and avoids any duplicated/repeated values for *Student_id* column. Inserting a repeated value again would result in an error.

TYPES OF KEY CONSTRAINTS

1. Primary Key Constraints

- It states that the primary key attributes are required to be unique and not null. That is, primary key attributes of a relation must not have null values and primary key attributes of two tuples must never be same.
- This constraint is specified on database schema to the primary key attributes to avoid duplications such that no two tuples are same.
- **Unique Values:** Each entry must be unique.
- **Not NULL:** No entry of NULL values.
- **Invalid:** Inserting a NULL value would result in an error.

Example of Primary Key Constraint

Student_id	Name	Semester	Age
101	Ramesh	5th	20
102	Kamlesh	5th	21
103	Akash	5th	22

Here, in the example the *Student_id* is the primary key attribute. The data entry of 4th tuple violates the primary key constraint that is specifies on the database schema and therefore this instance of database is not a legal instance.

- **Unique Values:** Each *Student_id* must be unique. 101, 102, 103 are valid. Inserting 101 again would result in an error.
- **Not NULL:** *Student_id* cannot be NULL. Here, *Student_id* values are not NULL.
- **Invalid:** A row with NULL for *Student_id* will be rejected.

TYPES OF KEY CONSTRAINTS

2. Unique Key Constraints

- The Unique key constraint in DBMS ensures that all values in a specified column (or group of columns) are distinct across the table.
- It prevents duplicate entries, maintaining data integrity, but unlike the primary key, it allows one NULL value.

Unique Values: The email column must contain unique values. A repeated value would result in an error.

Allows one NULL: Unique attribute column can contain one NULL value.

Valid: Inserting one NULL value for a unique attribute is valid .

Invalid: Inserting another NULL value for the same unique attribute column will be rejected.

Example of Unique Key Constraint

Employee_ID	Email	Name
1	aniket@example.com	Aniket Kumar
2	NULL	Shashwat Dubey
3	shashwat@example.com	Manvendra Sharma

Here, in the example the *Email* column has NULL value in 2nd record.

Unique Values: *Email* is a unique attribute. In this column, the values [aniket@example.com](#) and [shashwat@example.com](#) are valid. Adding another row with [aniket@example.com](#) again would result in an error.

Allows one NULL: *Email* is a unique attribute column containing one NULL value.

Valid: NULL in the second row of *Email* column is valid

Invalid: Adding another row with NULL in *Email* column will be rejected.

TYPES OF CONSTRAINTS

Referential Integrity Constraints

- Referential integrity constraints are rules that ensure relationships between tables remain consistent.
- They enforce that a **foreign key** in one table must either match a value in the referenced primary key of another table or be NULL.
- This guarantees the logical connection between related tables in a relational database.

Referential Integrity Constraints

Why Referential Integrity Constraints Are Important ?

Maintains consistency: Ensures relationships between tables are valid.

Prevents orphan records: Avoids cases where a record in a child table references a non-existent parent record.

Enforces logical relationships: Strengthens the logical structure of a relational database.

Statement of the constraint

The value in the Foreign Key (FK) column/columns of the **referencing relation** can be either:

- 1) *a value of an existing primary key value of a corresponding Primary Key PK in the **referenced relation**, or*
- 2) *a **null**.*

Example of Referential Integrity Constraints

Consider a table BLOCK containing Block_No and Block Location as the attributes where, Block_No is a Primary Key.

Block_No	Block Location
20	Chandigarh
21	Punjab
25	Delhi

This is another table containing Block_No as one of the attribute. Block_No attribute in this table is a Foreign Key attribute because it is referencing the primary key attribute of BLOCK table.

Student_id	Name	Semester	Block_No
22CSE101	Ramesh	5th	20
21CSE105	Kamlesh	6th	21
22CSE102	Aakash	5th	20
23CSE106	Mukesh	2nd	22

In the second table, the entry 22 under Block_No is not allowed because it is not present in the BLOCK table and hence throws an error. Therefore, only those values present in the BLOCK table and its repetition must be used else it violates the referential integrity.

Displaying a Relational Database Schema and its Constraints

- Each relation schema can be displayed as a row of attribute names.
- The name of the relation is written above the attribute names.
- The primary key attribute (or attributes) will be underlined.
- A foreign key (referential integrity) constraints is displayed as a directed arc (arrow) from the foreign key attributes to the referenced table.

Example: RELATIONAL SCHEMA for a COMPANY DATABASE

EMPLOYEE(Fname, Minit, Lname, Ssn, Badte, Address, Sex, Salary, Super_ssn, Dno)

DEPARTMENT(Dname, Dnumber, Mgr_ssn, Mgr_start_date)

DEPT_LOCATIONS(Dnumber, Dlocation)

PROJECT(Pname, Pnumber, Plocation, Dnum)

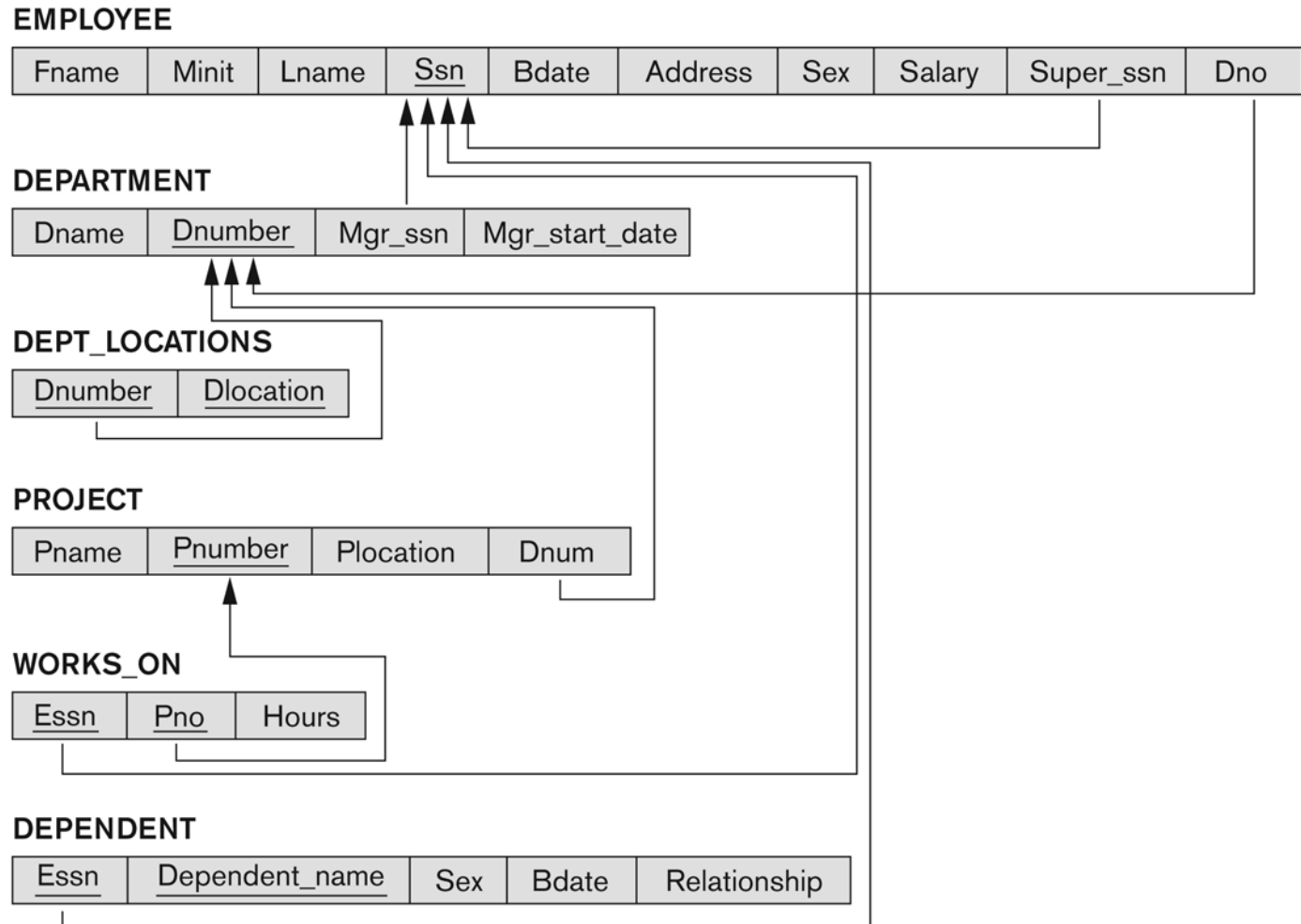
WORKS_ON(Essn, Pno, Hours)

DEPENDENT(Essn, Dependent_name, Sex, Bdate, Relationship)

RELATIONAL SCHEMA DIAGRAM

Figure 5.7

Referential integrity constraints displayed on the COMPANY relational database schema.



UPDATE OPERATIONS ON RELATIONS

- **INSERT** a tuple.
 - **DELETE** a tuple.
 - **MODIFY** a tuple.
-
- Integrity constraints should not be violated by the update operations.
 - Several update operations may have to be grouped together.
 - Updates may **propagate** to cause other updates automatically. Hence it is necessary to maintain integrity constraints.

UPDATE OPERATIONS ON RELATIONS

In case of integrity violation, several actions can be taken:

- Cancel the operation that causes the violation (RESTRICT or REJECT option)
- Perform the operation but inform the user of the violation
- Trigger additional updates so the violation is corrected (CASCADE option, SET NULL option)
- Execute a user-specified error-correction routine.

POSSIBLE VIOLATIONS FOR EACH OPERATION

INSERT may violate any of the constraints:

- ***Domain constraint:***

if one of the attribute values provided for the new tuple is not of the specified attribute domain.

- ***Key constraint:***

if the value of a key attribute in the new tuple already exists in another tuple in the relation

- ***Referential integrity:***

if a foreign key value in the new tuple references a primary key value that does not exist in the referenced relation.

- ***Entity integrity:***

if the primary key value is null in the new tuple

POSSIBLE VIOLATIONS FOR EACH OPERATION

DELETE may violate only referential integrity:

- If the primary key value of the tuple being deleted is referenced from other tuples in the database, can be remedied by several actions: RESTRICT, CASCADE, SET NULL.

RESTRICT option: reject the deletion

CASCADE option: propagate the new primary key value into the foreign keys of the referencing tuples

SET NULL option: set the foreign keys of the referencing tuples to NULL.

- One of the above options must be specified during database design for each foreign key constraint.

POSSIBLE VIOLATIONS FOR EACH OPERATION

UPDATE may violate domain constraint and NOT NULL constraint on an attribute being modified

- Any of the other constraints may also be violated, depending on the attribute being updated:

Updating the primary key (PK):

Similar to a DELETE followed by an INSERT

Need to specify similar options to DELETE

Updating a foreign key (FK):

May violate referential integrity

Updating an ordinary attribute (neither PK nor FK):

Can only violate domain constraints

CLASS EXERCISE

Taken from Exercise 5.15 of prescribed text book:

Consider the following relations for a database that keeps track of student enrollment in courses and the books adopted for each course:

STUDENT(SSN, Name, Major, Bdate)

COURSE(Course#, Cname, Dept)

ENROLL(SSN, Course#, Quarter, Grade)

BOOK_ADOPTION(Course#, Quarter,
Book_ISBN) TEXT(Book_ISBN, Book_Title,
Publisher, Author)

Draw a relational schema diagram specifying the foreign keys for this schema.

MODULE-2

Chapter-2: Schema Definition, Constraints, and Queries and Views

Reference: Fundamentals of Database Systems by Elmasri and Navathe

Data Definition, Constraints, and Schema Changes

- *Used to CREATE, DROP, and ALTER the descriptions of the tables (relations) of a database.*
- *These operations are carried out using CREATE, DROP and Alter commands of SQL.*

CREATE TABLE

- *Specifies a new base relation by giving it a name, and specifying each of its attributes and their data types (INTEGER, FLOAT, DECIMAL(i,j), CHAR(n), VARCHAR(n))*
- *A constraint NOT NULL may be specified on an attribute*

```
CREATE TABLE DEPARTMENT (  
    DNAME VARCHAR(10) NOT NULL,  
    DNUMBER INTEGER NOT NULL,  
    MGRSSN CHAR(9) ,  
    MGRSTARTDATE CHAR(9) );
```

CREATE TABLE

- *In SQL2, can use the CREATE TABLE command for specifying the primary key attributes, secondary keys, and referential integrity constraints (foreign keys).*
- *Key attributes can be specified via the PRIMARY KEY and UNIQUE phrases*

```
CREATE TABLE DEPT (  
    DNAME                VARCHAR(10)        NOT NULL,  
    DNUMBER             INTEGER             NOT NULL,  
    MGRSSN              CHAR(9) ,  
    MGRSTARTDATE       CHAR(9) ,  
    PRIMARY KEY (DNUMBER) ,  
    UNIQUE (DNAME) ,  
    FOREIGN KEY (MGRSSN) REFERENCES EMP ) ;
```

DROP TABLE

- *Used to remove a relation (base table) and its definition*
- *The relation can no longer be used in queries, updates, or any other commands since its description no longer exists*
- *Example:*

DROP TABLE DEPENDENT;

ALTER TABLE

- *Used to add an attribute to one of the base relations*
 - The new attribute will have NULLs in all the tuples of the relation right after the command is executed; hence, the NOT NULL constraint is not allowed for such an attribute
- *Example:*
***ALTER TABLE EMPLOYEE ADD JOB
VARCHAR(12) ;***
- *The database users must still enter a value for the new attribute JOB for each EMPLOYEE tuple.*
 - This can be done using the UPDATE command.

- *We can specify RESTRICT, CASCADE, SET NULL or SET DEFAULT on referential integrity constraints (foreign keys)*

```
CREATE TABLE DEPT (  
    DNAME          VARCHAR(10) NOT NULL,  
    DNUMBER        INTEGER          NOT NULL,  
    MGRSSN         CHAR(9) ,  
    MGRSTARTDATE   CHAR(9) ,  
    PRIMARY KEY (DNUMBER) ,  
    UNIQUE (DNAME) ,  
    FOREIGN KEY (MGRSSN) REFERENCES EMP  
    ON DELETE SET DEFAULT ON UPDATE CASCADE) ;
```

REFERENTIAL INTEGRITY OPTIONS

(Contd.)

```
CREATE TABLE EMP (  
    ENAME          VARCHAR(30)      NOT NULL,  
    ESSN           CHAR(9) ,  
    BDATE          DATE ,  
    DNO            INTEGER  DEFAULT 1 ,  
    SUPERSSN       CHAR(9) ,  
    PRIMARY KEY (ESSN) ,  
    FOREIGN KEY (DNO) REFERENCES DEPT  
        ON DELETE SET DEFAULT ON UPDATE  
        CASCADE ,  
    FOREIGN KEY (SUPERSSN) REFERENCES EMP  
        ON DELETE SET NULL ON UPDATE CASCADE) ;
```

Additional Data Types in SQL2 and SQL-99

Has DATE, TIME, and TIMESTAMP data types

- **DATE:**

- Made up of year-month-day in the format yyyy-mm-dd

- **TIME:**

- Made up of hour:minute:second in the format hh:mm:ss

- **TIME(i):**

- Made up of hour:minute:second plus i additional digits specifying fractions of a second
- format is hh:mm:ss:ii...i

Additional Data Types in SQL2 and SQL-99 (Contd.)

- ***TIMESTAMP:***
 - Has both DATE and TIME components
- ***INTERVAL:***
 - Specifies a relative value rather than an absolute value
 - Can be DAY/TIME intervals or YEAR/MONTH intervals
 - Can be positive or negative when added to or subtracted from an absolute value, the result is an absolute value

Retrieval Queries in SQL

- *SQL has one basic statement for retrieving information from a database; the **SELECT** statement*
 - This is *not the same* as the SELECT operation of the relational algebra
- *Important distinction between SQL and the formal relational model:*
 - SQL allows a table (relation) to have two or more tuples that are identical in all their attribute values
 - Hence, an SQL relation (table) is a **multi-set** (sometimes called a **bag**) of tuples; it is *not* a set of tuples
- *SQL relations can be constrained to be sets by specifying **PRIMARY KEY** or **UNIQUE** attributes, or by using the **DISTINCT** option in a query*

Retrieval Queries in SQL (Contd.)

- A **bag** or **multi-set** is like a set, but an element may appear more than once.
 - Example: $\{A, B, C, A\}$ is a bag. $\{A, B, C\}$ is also a bag that also is a set.
 - Bags also resemble lists, but the order is irrelevant in a bag.
- *Example:*
 - $\{A, B, A\} = \{B, A, A\}$ as bags
 - However, $[A, B, A]$ is not equal to $[B, A, A]$ as lists

Retrieval Queries in SQL (Contd.)

- *Basic form of the SQL SELECT statement is called a mapping or a SELECT-FROM-WHERE block*

SELECT <attribute list>
FROM <table list>
WHERE <condition>

- <attribute list> is a list of attribute names whose values are to be retrieved by the query
- <table list> is a list of the relation names required to process the query
- <condition> is a conditional (Boolean) expression that identifies the tuples to be retrieved by the query

Relational Database Schema--Figure 5.5

EMPLOYEE

FNAME	MINIT	LNAME	<u>SSN</u>	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
-------	-------	-------	------------	-------	---------	-----	--------	----------	-----

DEPARTMENT

DNAME	<u>DNUMBER</u>	MGRSSN	MGRSTARTDATE
-------	----------------	--------	--------------

DEPT_LOCATIONS

<u>DNUMBER</u>	<u>DLOCATION</u>
----------------	------------------

PROJECT

PNAME	<u>PNUMBER</u>	PLOCATION	DNUM
-------	----------------	-----------	------

WORKS_ON

<u>ESSN</u>	<u>PNO</u>	HOURS
-------------	------------	-------

DEPENDENT

<u>ESSN</u>	<u>DEPENDENT_NAME</u>	SEX	BDATE	RELATIONSHIP
-------------	-----------------------	-----	-------	--------------

Populated Database--Fig.5.6

EMPLOYEE	FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
	John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
	Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
	Alicia	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321	4
	Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
	Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
	Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
	Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
	James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	null	1

DEPARTMENT	DNAME	DNUMBER	MGRSSN	MGRSTARTDATE	DEPT_LOCATIONS	DNUMBER	DLOCATION
	Research	5	333445555	1988-05-22		1	Houston
	Administration	4	987654321	1995-01-01		4	Stafford
	Headquarters	1	888665555	1981-06-19		5	Bellaire
						5	Sugarland
						5	Houston

WORKS_ON	ESSN	PNO	HOURS
	123456789	1	32.5
	123456789	2	7.5
	666884444	3	40.0
	453453453	1	20.0
	453453453	2	20.0
	333445555	2	10.0
	333445555	3	10.0
	333445555	10	10.0
	333445555	20	10.0
	999887777	30	30.0
	999887777	10	10.0
	987987987	10	35.0
	987987987	30	5.0
	987654321	30	20.0
	987654321	20	15.0
	888665555	20	null

PROJECT	PNAME	PNUMBER	PLOCATION	DNUM
	ProductX	1	Bellaire	5
	ProductY	2	Sugarland	5
	ProductZ	3	Houston	5
	Computerization	10	Stafford	4
	Reorganization	20	Houston	1
	Newbenefits	30	Stafford	4

DEPENDENT	ESSN	DEPENDENT_NAME	SEX	BDATE	RELATIONSHIP
	333445555	Alice	F	1986-04-05	DAUGHTER
	333445555	Theodore	M	1983-10-25	SON
	333445555	Joy	F	1958-05-03	SPOUSE
	987654321	Abner	M	1942-02-28	SPOUSE
	123456789	Michael	M	1988-01-04	SON
	123456789	Alice	F	1988-12-30	DAUGHTER
	123456789	Elizabeth	F	1967-05-05	SPOUSE

Simple SQL Queries

- *Basic SQL queries correspond to using the following operations of the relational algebra:*
 - SELECT
 - PROJECT
 - JOIN
- *All subsequent examples use the COMPANY database*

Simple SQL Queries

(Contd.)

- *Example of a simple query on one relation*
- *Query 0: Retrieve the birthdate and address of the employee whose name is 'John B. Smith'.*

```
Q0:SELECT      BDATE, ADDRESS  
      FROM      EMPLOYEE  
      WHERE     FNAME='John' AND MINIT='B'  
      AND       LNAME='Smith'
```

- Similar to a SELECT-PROJECT pair of relational algebra operations:
 - The SELECT-clause specifies the projection attributes and the WHERE-clause specifies the selection condition
- However, the result of the query may contain duplicate tuples

Simple SQL Queries

- *Query 1: Retrieve the name and address of all employees who work for the 'Research' department.*

```
Q1:SELECT      FNAME, LNAME, ADDRESS  
      FROM      EMPLOYEE, DEPARTMENT  
      WHERE      DNAME='Research' AND DNUMBER=DNO
```

- Similar to a SELECT-PROJECT-JOIN sequence of relational algebra operations
- (DNAME='Research') is a selection condition (corresponds to a SELECT operation in relational algebra)
- (DNUMBER=DNO) is a join condition (corresponds to a JOIN operation in relational algebra)

Simple SQL Queries

- *Query 2: For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birthdate.*

*Q2: SELECT PNUMBER, DNUM, LNAME, BDATE, ADDRESS
 FROM PROJECT, DEPARTMENT, EMPLOYEE
 WHERE DNUM=DNUMBER AND MGRSSN=SSN
 AND PLOCATION='Stafford'*

- In Q2, there are two join conditions
- The join condition DNUM=DNUMBER relates a project to its controlling department
- The join condition MGRSSN=SSN relates the controlling department to the employee who manages that department

Aliases, * and DISTINCT, Empty WHERE-clause

- *In SQL, we can use the same name for two (or more) attributes as long as the attributes are in different relations*
- *A query that refers to two or more attributes with the same name must qualify the attribute name with the relation name by prefixing the relation name to the attribute name*
- *Example:*
- ***EMPLOYEE.LNAME, DEPARTMENT.DNAME***

ALIASES

- *Some queries need to refer to the same relation twice*
 - In this case, *aliases* are given to the relation name
- *Query 8: For each employee, retrieve the employee's name, and the name of his or her immediate supervisor.*

```
Q8:  SELECT      E.FNAME, E.LNAME, S.FNAME, S.LNAME
      FROM        EMPLOYEE E S
      WHERE       E.SUPERSSN=S.SSN
```

- In Q8, the alternate relation names E and S are called *aliases* or *tuple variables* for the EMPLOYEE relation
- We can think of E and S as two different *copies* of EMPLOYEE; E represents employees in role of *supervisees* and S represents employees in role of *supervisors*

ALIASES (contd.)

- *Aliasing can also be used in any SQL query for convenience*
- *Can also use the AS keyword to specify aliases*

Q8: SELECT E.FNAME, E.LNAME,
 S.FNAME, S.LNAME
FROM EMPLOYEE AS E,
 EMPLOYEE AS S
WHERE E.SUPERSSN=S.SSN

UNSPECIFIED WHERE-clause

- *A missing WHERE-clause indicates no condition; hence, all tuples of the relations in the FROM-clause are selected*
 - This is equivalent to the condition WHERE TRUE
- *Query 9: Retrieve the SSN values for all employees.*
 - Q9: SELECT SSN
 FROM EMPLOYEE
- *If more than one relation is specified in the FROM-clause and there is no join condition, then the CARTESIAN PRODUCT of tuples is selected*

UNSPECIFIED WHERE-clause (contd.)

- *Example:*

Q10: SELECT SSN, DNAME
 FROM EMPLOYEE, DEPARTMENT

- It is extremely important not to overlook specifying any selection and join conditions in the WHERE-clause; otherwise, incorrect and very large relations may result

USE OF *

- *To retrieve all the attribute values of the selected tuples, a * is used, which stands for all the attributes*

Examples:

Q1C:	SELECT	*
	FROM	EMPLOYEE
	WHERE	DNO=5

Q1D:	SELECT	*
	FROM	EMPLOYEE, DEPARTMENT
	WHERE	DNAME='Research' AND DNO=DNUMBER

USE OF DISTINCT

- *SQL does not treat a relation as a set; duplicate tuples can appear*
- *To eliminate duplicate tuples in a query result, the keyword **DISTINCT** is used*
- *For example, the result of Q11 may have duplicate SALARY values whereas Q11A does not have any duplicate values*

Q11: SELECT SALARY
FROM EMPLOYEE

Q11A: SELECT **DISTINCT** SALARY
FROM EMPLOYEE

SET OPERATIONS (contd.)

- SQL has directly incorporated some set operations
- There is a union operation (UNION), and in some versions of SQL there are set difference (MINUS) and intersection (INTERSECT) operations
- The resulting relations of these set operations are sets of tuples; duplicate tuples are eliminated from the result
- The set operations apply only to union compatible relations; the two relations must have the same attributes and the attributes must appear in the same order

SET OPERATIONS (contd.)

- *Query 4: Make a list of all project numbers for projects that involve an employee whose last name is 'Smith' as a worker or as a manager of the department that controls the project.*

```
Q4:      (SELECT      PNAME
          FROM        PROJECT, DEPARTMENT,
                      EMPLOYEE
          WHERE       DNUM=DNUMBER AND
                      MGRSSN=SSN AND LNAME='Smith')

          UNION
          (SELECT      PNAME
          FROM        PROJECT, WORKS_ON, EMPLOYEE
          WHERE       PNUMBER=PNO AND
                      ESSN=SSN AND NAME='Smith')
```

NESTING OF QUERIES

- *A complete SELECT query, called a nested query, can be specified within the WHERE-clause of another query, called the outer query*
 - Many of the previous queries can be specified in an alternative form using nesting
- *Query 1: Retrieve the name and address of all employees who work for the 'Research' department.*

```
Q1:SELECT      FNAME, LNAME, ADDRESS
      FROM      EMPLOYEE
      WHERE     DNO IN (SELECT DNUMBER
      FROM      DEPARTMENT
      WHERE     DNAME='Research' )
```

NESTING OF QUERIES (contd.)

- *The nested query selects the number of the 'Research' department*
- *The outer query select an EMPLOYEE tuple if its DNO value is in the result of either nested query*
- *The comparison operator IN compares a value v with a set (or multi-set) of values V , and evaluates to TRUE if v is one of the elements in V*
- *In general, we can have several levels of nested queries*
- *A reference to an unqualified attribute refers to the relation declared in the innermost nested query*
- *In this example, the nested query is not correlated with the outer query*

CORRELATED NESTED QUERIES

- *If a condition in the WHERE-clause of a nested query references an attribute of a relation declared in the outer query, the two queries are said to be correlated*
 - The result of a correlated nested query is different for each tuple (or combination of tuples) of the relation(s) the outer query
- *Query 12: Retrieve the name of each employee who has a dependent with the same first name as the employee.*

```
Q12: SELECT      E.FNAME, E.LNAME
      FROM        EMPLOYEE AS E
      WHERE       E.SSN IN
                  (SELECT      ESSN
                   FROM        DEPENDENT
                   WHERE       ESSN=E.SSN AND
                              E.FNAME=DEPENDENT_NAME)
```

CORRELATED NESTED QUERIES

(contd.)

- *In Q12, the nested query has a different result in the outer query*
- *A query written with nested SELECT... FROM... WHERE... blocks and using the = or IN comparison operators can **always** be expressed as a single block query. For example, Q12 may be written as in Q12A*

Q12A:	SELECT	E.FNAME, E.LNAME
	FROM	EMPLOYEE E, DEPENDENT D
	WHERE	E.SSN=D.ESSN AND
		E.FNAME=D.DEPENDENT_NAME

CORRELATED NESTED QUERIES

(contd.)

- *The original SQL as specified for SYSTEM R also had a **CONTAINS** comparison operator, which is used in conjunction with nested correlated queries*
 - This operator was *dropped from the language*, possibly because of the difficulty in implementing it efficiently
 - Most implementations of SQL do not have this operator
 - The CONTAINS operator compares *two sets of values*, and returns TRUE if one set contains all values in the other set
 - Reminiscent of the division operation of algebra

CORRELATED NESTED QUERIES

(contd.)

- *Query 3: Retrieve the name of each employee who works on all the projects controlled by department number 5.*

```
Q3:      SELECT      FNAME, LNAME
          FROM        EMPLOYEE
          WHERE (      (SELECT      PNO
                        FROM        WORKS_ON
                        WHERE        SSN=ESSN)
                    CONTAINS
                        (SELECT      PNUMBER
                        FROM        PROJECT
                        WHERE        DNUM=5) )
```


CORRELATED NESTED QUERIES (contd.)

- In Q3, the second nested query, which is not correlated with the outer query, retrieves the project numbers of all projects controlled by department 5
- The first nested query, which is correlated, retrieves the project numbers on which the employee works, which is different for each employee tuple because of the correlation

THE EXISTS FUNCTION (contd.)

- *EXISTS is used to check whether the result of a correlated nested query is empty (contains no tuples) or not.*
 - We can formulate Query 12 in an alternative form that uses EXISTS as Q12B

THE EXISTS FUNCTION (contd.)

- *Query 12: Retrieve the name of each employee who has a dependent with the same first name as the employee.*

```
Q12B:      SELECT      FNAME, LNAME
            FROM        EMPLOYEE
            WHERE        EXISTS (SELECT      *
                                FROM        DEPENDENT
                                WHERE        SSN=ESSN
                                AND
                                FNAME=DEPENDENT_NAME)
```

THE EXISTS FUNCTION (contd.)

- *Query 6: Retrieve the names of employees who have no dependents.*

```
Q6:      SELECT      FNAME, LNAME
          FROM        EMPLOYEE
          WHERE        NOT EXISTS (SELECT      *
                                   FROM        DEPENDENT
                                   WHERE        SSN=ESSN)
```

- *In Q6, the correlated nested query retrieves all DEPENDENT tuples related to an EMPLOYEE tuple. If none exist, the EMPLOYEE tuple is selected*
 - EXISTS is necessary for the expressive power of SQL

EXPLICIT SETS

- *It is also possible to use an **explicit (enumerated) set of values** in the **WHERE**-clause rather than a nested query*

Query 13: Retrieve the social security numbers of all employees who work on project number 1, 2, or 3.

```
Q13:      SELECT      DISTINCT ESSN
           FROM  WORKS_ON
           WHERE      PNO IN (1, 2, 3)
```

NULLS IN SQL QUERIES

- *SQL allows queries that check if a value is **NULL** (missing or undefined or not applicable)*
- *SQL uses **IS** or **IS NOT** to compare NULLs because it considers each NULL value distinct from other NULL values, so equality comparison is not appropriate.*
- *Query 14: Retrieve the names of all employees who do not have supervisors.*

Q14: SELECT FNAME, LNAME
 FROM EMPLOYEE
 WHERE SUPERSSN IS NULL

- *Note: If a join condition is specified, tuples with NULL values for the join attributes are not included in the result*

Joined Relations Feature in SQL2

- *Can specify a "joined relation" in the FROM-clause*
 - Looks like any other relation but is the result of a join
 - Allows the user to specify different types of joins (regular "theta" JOIN, NATURAL JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, CROSS JOIN, etc)

Joined Relations Feature in SQL2 (contd.)

- *Examples:*

Q8:SELECT	E.FNAME, E.LNAME, S.FNAME, S.LNAME
FROM	EMPLOYEE E S
WHERE	E.SUPERSSN=S.SSN

- *can be written as:*

Q8:SELECT	E.FNAME, E.LNAME, S.FNAME, S.LNAME
FROM	(EMPLOYEE E LEFT OUTER JOIN
	EMPLOYEES ON E.SUPERSSN=S.SSN)

Joined Relations Feature in SQL2

(contd.)

- *Examples:*

```
Q1:SELECT      FNAME, LNAME, ADDRESS
      FROM EMPLOYEE, DEPARTMENT
      WHERE      DNAME='Research' AND DNUMBER=DNO
```

- *could be written as:*

```
Q1:SELECT      FNAME, LNAME, ADDRESS
      FROM      (EMPLOYEE JOIN DEPARTMENT
                  ON DNUMBER=DNO)
      WHERE      DNAME='Research'
```

- *or as:*

```
Q1:SELECT      FNAME, LNAME, ADDRESS
      FROM      (EMPLOYEE NATURAL JOIN
                  DEPARTMENT
                  AS DEPT(DNAME, DNO, MSSN, MSDATE))
      WHERE      DNAME='Research'
```

Joined Relations Feature in SQL2

(contd.)

- *Another Example: Q2 could be written as follows; this illustrates multiple joins in the joined tables*

```
Q2: SELECT      PNUMBER, DNUM, LNAME,  
                BDATE, ADDRESS  
FROM (PROJECT JOIN  
      DEPARTMENT ON  
      DNUM=DNUMBER) JOIN  
EMPLOYEE ON  
MGRSSN=SSN) )  
WHERE          PLOCATION='Stafford'
```

AGGREGATE FUNCTIONS (contd.)

- Include **COUNT, SUM, MAX, MIN, and AVG**
- *Query 15: Find the maximum salary, the minimum salary, and the average salary among all employees.*
Q15: SELECT MAX(SALARY),
 MIN(SALARY), AVG(SALARY)
 FROM EMPLOYEE
- *Some SQL implementations may not allow more than one function in the SELECT-clause*

AGGREGATE FUNCTIONS (contd.)

- *Query 16: Find the maximum salary, the minimum salary, and the average salary among employees who work for the 'Research' department.*

```
Q16:      SELECT      MAX(SALARY),  
                  MIN(SALARY), AVG(SALARY)  
FROM  EMPLOYEE, DEPARTMENT  
WHERE      DNO=DNUMBER AND  
           DNAME='Research'
```

AGGREGATE FUNCTIONS (contd.)

- *Queries 17 and 18: Retrieve the total number of employees in the company (Q17), and the number of employees in the 'Research' department (Q18).*

Q17: SELECT COUNT (*)
 FROM EMPLOYEE

Q18: SELECT COUNT (*)
 FROM EMPLOYEE, DEPARTMENT
 WHERE DNO=DNUMBER AND
 DNAME='Research'

GROUPING

- In many cases, we want to apply the aggregate functions to subgroups of tuples in a relation
- Each subgroup of tuples consists of the set of tuples that have the same value for the grouping attribute(s)
- The function is applied to each subgroup independently
- SQL has a **GROUP BY**-clause for specifying the grouping attributes, which must also appear in the SELECT-clause

GROUPING (contd.)

- *Query 20: For each department, retrieve the department number, the number of employees in the department, and their average salary.*

```
Q20:      SELECT      DNO, COUNT (*), AVG (SALARY)
           FROM        EMPLOYEE
           GROUP BY    DNO
```

- In Q20, the EMPLOYEE tuples are divided into groups-
 - Each group having the same value for the grouping attribute DNO
- The COUNT and AVG functions are applied to each such group of tuples separately
- The SELECT-clause includes only the grouping attribute and the functions to be applied on each group of tuples
- A join condition can be used in conjunction with grouping

GROUPING (contd.)

- *Query 21: For each project, retrieve the project number, project name, and the number of employees who work on that project.*

```
Q21:      SELECT      PNUMBER, PNAME, COUNT (*)  
          FROM        PROJECT, WORKS_ON  
          WHERE        PNUMBER=PNO  
          GROUP BY    PNUMBER, PNAME
```

- In this case, the grouping and functions are applied after the joining of the two relations

THE HAVING-CLAUSE

- Sometimes we want to retrieve the values of these functions for only those groups that satisfy certain conditions
- The **HAVING**-clause is used for specifying a selection condition on groups (rather than on individual tuples)

THE HAVING-CLAUSE (contd.)

- *Query 22: For each project on which more than two employees work, retrieve the project number, project name, and the number of employees who work on that project.*

```
Q22:      SELECT      PNUMBER, PNAME,
                COUNT(*)
FROM  PROJECT, WORKS_ON
WHERE      PNUMBER=PNO
GROUP BY   PNUMBER, PNAME
HAVING     COUNT (*) > 2
```

SUBSTRING COMPARISON

- The **LIKE** comparison operator is used to compare partial strings
- Two reserved characters are used: '%' (or '*' in some implementations) replaces an arbitrary number of characters, and '_' replaces a single arbitrary character

SUBSTRING COMPARISON (contd.)

- *Query 25: Retrieve all employees whose address is in Houston, Texas. Here, the value of the ADDRESS attribute must contain the substring 'Houston,TX' in it.*

```
Q25:      SELECT      FNAME, LNAME
           FROM  EMPLOYEE
           WHERE      ADDRESS LIKE
                     '%Houston,TX%'
```

SUBSTRING COMPARISON (contd.)

- *Query 26: Retrieve all employees who were born during the 1950s.*
 - Here, '5' must be the 8th character of the string (according to our format for date), so the BDATE value is '_____5_', with each underscore as a place holder for a single arbitrary character.

Q26: SELECT FNAME, LNAME
 FROM EMPLOYEE
 WHERE BDATE LIKE '_____5_'

- *The LIKE operator allows us to get around the fact that each value is considered atomic and indivisible*
 - Hence, in SQL, character string attribute values are not atomic

ARITHMETIC OPERATIONS

- *The standard arithmetic operators '+', '-', '*', and '/' (for addition, subtraction, multiplication, and division, respectively) can be applied to numeric values in an SQL query result*
- *Query 27: Show the effect of giving all employees who work on the 'ProductX' project a 10% raise.*

```
Q27:      SELECT      FNAME, LNAME, 1.1*SALARY
           FROM        EMPLOYEE, WORKS_ON,
           PROJECT
           WHERE        SSN=ESSN AND PNO=PNUMBER
                      AND PNAME='ProductX'
```

ORDER BY

- The **ORDER BY** clause is used to sort the tuples in a query result based on the values of some attribute(s)
- Query 28: Retrieve a list of employees and the projects each works in, ordered by the employee's department, and within each department ordered alphabetically by employee last name.

```
Q28:      SELECT      DNAME, LNAME, FNAME, PNAME
           FROM        DEPARTMENT, EMPLOYEE,
                       WORKS_ON, PROJECT
           WHERE        DNUMBER=DNO AND SSN=ESSN
                       AND PNO=PNUMBER
           ORDER BY    DNAME, LNAME
```

ORDER BY (contd.)

- The default order is in ascending order of values.
- We can specify the keyword **DESC** if we want a descending order; the keyword **ASC** can be used to explicitly specify ascending order, even though it is the default

Summary of SQL Queries

- *A query in SQL can consist of up to six clauses, but only the first two, **SELECT** and **FROM**, are mandatory. The clauses are specified in the following order:*

SELECT	<i><attribute list></i>
FROM	<i><table list></i>
[WHERE	<i><condition>]</i>
[GROUP BY	<i><grouping attribute(s)>]</i>
[HAVING	<i><group condition>]</i>
[ORDER BY	<i><attribute list>]</i>

Summary of SQL Queries (contd.)

- *The SELECT-clause lists the attributes or functions to be retrieved*
- *The FROM-clause specifies all relations (or aliases) needed in the query but not those needed in nested queries*
- *The WHERE-clause specifies the conditions for selection and join of tuples from the relations specified in the FROM-clause*
- *GROUP BY specifies grouping attributes*
- *HAVING specifies a condition for selection of groups*
- *ORDER BY specifies an order for displaying the result of a query*
 - A query is evaluated by first applying the WHERE-clause, then GROUP BY and HAVING, and finally the SELECT-clause

Specifying Updates in SQL

- *There are three SQL commands to modify the database:*
INSERT,
DELETE, and
UPDATE

INSERT

- In its simplest form, it is used to add one or more tuples to a relation
- Attribute values should be listed in the same order as the attributes were specified in the **CREATE TABLE** command

INSERT (contd.)

- *Example:*

```
U1:INSERT INTO      EMPLOYEE
      VALUES ('Richard','K','Marini', '653298653', '30-DEC-52',
'98 Oak Forest,Katy,TX', 'M', 37000,'987654321', 4 )
```

- *An alternate form of INSERT specifies explicitly the attribute names that correspond to the values in the new tuple*

- Attributes with NULL values can be left out

- *Example: Insert a tuple for a new EMPLOYEE for whom we only know the FNAME, LNAME, and SSN attributes.*

```
U1A:  INSERT INTO      EMPLOYEE (FNAME, LNAME,
                                   SSN)
      VALUES ('Richard', 'Marini', '653298653')
```

INSERT (contd.)

- *Important Note: Only the constraints specified in the DDL commands are automatically enforced by the DBMS when updates are applied to the database*
- Another variation of INSERT allows insertion of *multiple tuples* resulting from a query into a relation.

INSERT (contd.)

- *Example: Suppose we want to create a temporary table that has the name, number of employees, and total salaries for each department.*
 - A table DEPTS_INFO is created by U3A, and is loaded with the summary information retrieved from the database by the query in U3B.

```
U3A:      CREATE TABLE  DEPTS_INFO
              (DEPT_NAME      VARCHAR(10),
               NO_OF_EMPS     INTEGER,
               TOTAL_SAL      INTEGER);
```

```
U3B:      INSERT INTO    DEPTS_INFO (DEPT_NAME,
                                NO_OF_EMPS, TOTAL_SAL)
              SELECT      DNAME, COUNT (*), SUM (SALARY)
              FROM         DEPARTMENT, EMPLOYEE
              WHERE        DNUMBER=DNO
              GROUP BY     DNAME ;
```

INSERT (contd.)



Note: The DEPTS_INFO table may not be up-to-date if we change the tuples in either the DEPARTMENT or the EMPLOYEE relations after issuing U3B. We have to create a view (see later) to keep such a table up to date.

DELETE

- *Removes tuples from a relation*
 - Includes a WHERE-clause to select the tuples to be deleted
 - Referential integrity should be enforced
 - Tuples are deleted from only *one table* at a time (unless CASCADE is specified on a referential integrity constraint)
 - A missing WHERE-clause specifies that *all tuples* in the relation are to be deleted; the table then becomes an empty table
 - The number of tuples deleted depends on the number of tuples in the relation that satisfy the WHERE-clause

DELETE (contd.)

- *Examples:*

U4A: DELETE FROM EMPLOYEE
 WHERE LNAME='Brown'

U4B: DELETE FROM EMPLOYEE
 WHERE SSN='123456789'

U4C: DELETE FROM EMPLOYEE
 WHERE DNO IN
 (SELECT DNUMBER
 FROM DEPARTMENT
 WHERE
 DNAME='Research')

U4D: DELETE FROM EMPLOYEE

UPDATE

- Used to modify attribute values of one or more selected tuples
- A WHERE-clause selects the tuples to be modified
- An additional SET-clause specifies the attributes to be modified and their new values
- Each command modifies tuples in the same relation.
- Referential integrity should be enforced

UPDATE (contd.)

- *Example: Change the location and controlling department number of project number 10 to 'Bellaire' and 5, respectively.*

```
U5: UPDATE      PROJECT
      SET        PLOCATION = 'Bellaire',
                DNUM = 5
      WHERE      PNUMBER=10
```

UPDATE (contd.)

- *Example: Give all employees in the 'Research' department a 10% raise in salary.*

```
U6:UPDATE      EMPLOYEE
      SET       SALARY = SALARY *1.1
      WHERE     DNO IN (SELECT  DNUMBER
                          FROM    DEPARTMENT
                          WHERE    DNAME='Research')
```

- *In this request, the modified SALARY value depends on the original SALARY value in each tuple*
 - The reference to the SALARY attribute on the right of = refers to the old SALARY value before modification
 - The reference to the SALARY attribute on the left of = refers to the new SALARY value after modification

Recap of SQL Queries

- *A query in SQL can consist of up to six clauses, but only the first two, **SELECT** and **FROM**, are mandatory. The clauses are specified in the following order:*

SELECT <attribute list>

FROM <table list>

[WHERE <condition>]

[GROUP BY <grouping attribute(s)>]

[HAVING <group condition>]

[ORDER BY <attribute list>]

- *There are three SQL commands to modify the database: **INSERT**, **DELETE**, and **UPDATE***